1 **Design & Implementation of Human-Computer Interfaces**
NPTEL-MOOCS

Dr Samit Bhattacharya
Computer Science and Engineering
IIT Guwahati

2 **L28 - Black Box Testing (II)**

Dr Samit Bhattacharya
Computer Science and Engineering
IIT Guwahati

3

4 **Recap**

- Code testing - TWO ways
  - Review-based
  - Execution-based

5 **Execution Based Testing**

- Two types
  - Black box testing (functionality testing)
  - White box testing (structural testing)

6 **BBT - Idea**

- Design test cases based on input/output values ONLY [no knowledge of design or code required]

7 **Idea**

- TWO main approaches to design test cases
  - Equivalence class partitioning
  - Boundary value analysis

8 **Equivalence Class & Partitioning**

- Domain of input values partitioned into sets, each called 'equivalence class', such that program behaves the same for all the member of an (equivalent) class

9 **Equivalence Class & Partitioning**

- Testing code with any ONE value of an equivalence class is as good as testing with ALL input values belonging to that class

10 **Examples**

11 **Example 1**

- Consider a code to compute square root of an input integer in the range [0, 5000]
  - Input – integer within the range, output – square root
  - What is/are the equivalence class(es)?

12 **Obvious Choice**

- Let us define ONE equivalence class – the most obvious one!
  - Set of all integers within the range [0, 5000], both inclusive

13 **Example 1**

- Is that the best thing to do?
- Let's see

14 **Example 1**

- Recollect the definition of equivalence class – for all elements in the class, the program should behave in the same way
- So, we can take any one element from the set to test the code

15 **Example 1**

- What is our test suite then?
- It should be of the form <input, output>
  - Input = a number taken from the equivalence class
  - Output = the expected output of the program for the input

16 **Example 1**

- So, let's take one pair of input/output as the test suite
- E.g. <4, 2>

17 **Example 1**

- If we use only this single test case, are we going to get a comprehensive idea of the program behavior?
- Ideally, yes – we have taken a representative case from the equivalence class

18 **Example 1**

- Now suppose a user inputs -9!
- Is our program able to deal with this input in real usage scenario?

19 **Example 1**

- Answer – a big NO!
- We never tested for this input – it's outside the range we considered
- So, we missed something in our equivalence class formulation

20 **Next Obvious Choice**

- To take care, let's us redefine our equivalence class definition – now, the class is defined as "the set of all integers, both positive and negative" – another obvious one!
- Will it serve our purpose?

21 **Example 1**

- Answer is still NO!!

22 **Example 1**

- With this new definition, we take one test case
- Let us say, it is <9, 3>

- Since the entire domain of integers is now equivalence class, any one element (here, 4) should be representative and sufficient to capture program behavior

23 **Example 1**

- Clearly, that is NOT the case
- It still can't handle user inputs like -9.
- Also, we don't know what happens if user inputs 5001 onwards (outside the range)

24 **Note**

- So, we have to be very careful
- We shouldn't simply consider the most obvious range as the equivalence class
- It may affect our ability to form suitable test suit for comprehensive testing

25 **A Non-Obvious Solution**

- Now consider THREE equivalence classes
  - Instead of one earlier

26 **Example 1**

- Set 1 - Set of negative integers (INVALID input range)

27 **Example 1**

- Set 2 - set of integers in the range of [0, 5000], both inclusive (VALID input range)

28 **Example 1**

- Set 3 – set of ALL integers larger than 5000 (INVALID input range)

29 **Example 1**

- Test suite must include representative input (and corresponding output) for each of the three equivalence classes

30 **Example 1**

- e.g., a test suite: {(-5,output),(500,output),(6000,output)}
- Note
  - It is a set of THREE elements
  - Each element is a representative of the corresponding equivalence set (set 1, set 2 and set 3)
  - Each element of the form (input, output)

31 **Note**

- Thus, with a more granular classes, we could take care of different situations
- And accordingly modify the code to take care of those

32

33 **Example 2**

- Let's consider another program for testing
- It computes intersection point of two straight lines and displays the result

34 **Example 2**

- Input - two integer pairs (m1, c1) and (m2, c2);
- Each pair defines a straight line of the form y=mx + c
- Output – intersection point

- What is/are the equivalence class(es)?

35 **First (Obvious) Choice**

- Set of ALL lines!
- Clearly, it will not work
- Why?

36 **First (Obvious) Choice**

- For the same reason - we'll not be able to create a test suite to capture *variations* in input
- What variations we are looking at?

37 **Input Variation**

- Input is pair of straight line
- We should look to capture different *relationships* between the pair from the point of view of "intersection"

38 **Relationships**

- THREE possibilities

39 **Relationship**

- Parallel lines (m1=m2, c1≠c2)
- Intersecting lines (m1≠m2)
- Coincident lines (m1=m2, c1=c2)

40 **Equivalence Class**

- THREE
  - Set of all parallel lines
  - Set of all intersecting lines
  - Set of all coincidental lines

41 **Equivalence Class**

- At least ONE representative from each class
- Ex [(2, 2) (2, 5); (5, 5) (7, 7); (10, 10) (10, 10)]

42 **Food for Thought**

- What would be the equivalence classes if, instead of lines, we now consider line segments?
  - Input of the form: (x11,y11),(x21,y21); (x12,y12),(x22,y22) [each pair indicate one end point]
  - Output – intersection point or NULL

43

44 **Example 3**

- Let's consider a slightly different example
- You have developed an user authentication system, as part of the a bigger system. The authentication system/function takes as input user ID and produces success/failure message (which can be used to proceed further).

- Let's try to design the test suite for the example

45 **Test Suite Design**

- Input = user ID
  - Not sufficient, need more clarity

46 **First Thing – Clearly Specified Input**

- Clarified input – alphanumeric string
  - Still not sufficient
  - Need length information (string of any length acceptable)

- Suppose you specified "5 character string" as valid ID

47 **Test Suite Design**

- Obvious/intuitive solution (equivalence classes)
  - Set of all alphanumeric strings
  - Is it sufficient?

48 **Test Suite Design**

- NO!!
- Program behavior for a 10 character string and 5 character string should not be the same

49 **A Better Solution**

- Equivalence class 1 – set of all alphanumeric strings of length 5
- Equivalence class 2 – all other strings

- Will it work?

50 **An Even Better Solution**

- Class 1 – set of all 5 character alphabetic (only) strings
- Class 2 – set of all 5 character numeric (only) strings
- Class 3 – set of all 5 character alphanumeric strings
- Class 4 – set of all alphanumeric strings with length >5
- Class 5 – set of all alphanumeric strings with length < 5
- Class 6 – set of all strings with special characters

51 **Note**

- Test suite
  - One (input, output) pair from each set
  - At least six test cases
- Can we do any better (take home exercise)!!

52

53 **Boundary Value Problem**

- Programming error frequently occurs at the boundaries of equivalence classes
- Due to oversight - programmers fail to notice special processing required for inputs at class boundaries

- E.g., may improperly use < instead of <= or conversely <= for <

54 **Boundary Value Analysis**

- Selection of test cases at the boundaries (of equivalence classes)

55 **Boundary Value Analysis**

- Ex - reconsider function to compute square root of integer values in the range of 0 and 5000

56 **Boundary Value Analysis**

- Earlier we considered test cases for 3 classes
  - Set of negative integers
  - Set of integers in the range of [0, 5000]
  - Integers larger than 5000

57 **Boundary Value Analysis**

- Along with those, we should include test cases in our suit
  - <-1, output>
  - <0, output>
  - <5000, output>
  - <5001, output>

58

59 **Note**

- Equivalence classes are sets ☐ should use set notations to represent
- Test cases should contain both input and expected output
- Remember to include test case(s) from valid class(es), invalid class(es) and boundary cases in black-box test suites

60 **Note**

- Next lecture – case study

61 **Reference**

- Rajib Mall (2018). Fundamentals of Software Engineering, 5th ed, PHI Learning Pvt Ltd. Chapter 10
- Roger S Pressman (2020). Software Engineering: A Practitioner's Approach, 9th ed, McGraw-Hill Education, New York, Chapter 19-21